ERP-driven Financial Reporting: Controlled Agentic Architecture

Take-home design | 2024-Q4 | Functional currency USD | Prototype: manual adjustment validation

Executive decision

Use one workflow orchestrator with **deterministic accounting services** and an **optional constrained LLM assistant** for ambiguous account classification and plain-language explanations. No multi-agent swarm. The submitted runnable prototype implements a deterministic journal-validation slice and read-only input diagnostics, not financial statement generation or a deployed LLM. An auditor should never have to trust an LLM's arithmetic.

Clarifications requested before implementation

1. Which reporting standard (US GAAP/IFRS) and entity/consolidation perimeter apply? In particular, are opening TB figures balance-sheet-only, and do the current TB figures include year-to-date P&L?

2. Are adjustment amounts always functional-currency USD, and what is the required policy for historical, average, and closing rates, missing FX rates, and rounding tolerances?

3. Which identifiers define intercompany counterparty, legal entity, and elimination pair; who can approve unresolved mappings and exceptions?

Until answered, assumptions are explicit: adjustment amounts are USD per JSON metadata; zero tolerance for unbalanced individual journals; unresolved mapping and missing FX rate block affected downstream financial statements; intercompany entries need human review; no automatic correction.

Decomposition and topology

1. **Ingestion**: immutable raw files, content hashes, source row IDs, schema version, run ID, access controls.

2. **Normalization**: typed decimals, account and currency codes as strings, canonical journal schema; preserve original records.

3. **COA mapping**: exact approved mapping first, then optional LLM candidate suggestions with evidence/confidence and a human decision; never create an approved account automatically.

4. **Adjustment validator (implemented)**: account existence/postability, debit/credit parity, nonnegative finite amounts, two-sided lines, duplicate IDs, suspicious same-account offset, intercompany review.

5. **FX and consolidation**: select documented rates per monetary/nonmonetary account and accounting policy, enforce rate coverage, pair IC transactions by legal entity and counterparty, eliminate with balanced journals.

6. **Deterministic statement compiler**: build BS and P&L from mapped ledger; derive cash flow from supported movements and transaction metadata (not TB differences alone); SOCIE from equity rollforward and supporting transactions.

7. **Reconciliation and publication**: validate ledger debits=credits, A=L+E, P&L net income to equity rollforward, cash movement to cash flow, SOCIE closing to BS equity, comparative continuity, cross-entity elimination; publish only when required gates pass.

Flow: Raw inputs -> Ingest -> Normalize -> [COA suggestions + human approval] -> Adjustment validation -> FX/IC processing -> Deterministic statements -> Reconciliation -> Human release -> immutable reports + trace manifest. Failures go to exception queue, not silent repair.

LLM vs deterministic boundary

Code: money arithmetic using Decimal, date/rate selection, balanced-entry rules, referential integrity, graph checks, aggregations, formulae, immutable provenance, posting and release decisions.

LLM (proposed, not implemented): explain an exception using structured error codes; suggest candidate COA mappings with retrieved approved taxonomy; propose remediation for an accountant. Constrain output to a typed schema `{candidates:[{code,reason,evidence,confidence}], explanation}`. Confidence is a ranking heuristic, not a calibrated probability without evaluation. The LLM cannot post journals, approve mappings, invent FX rates, or waive validation gates.

Actual company-data findings

• `JE-002`: debit $28,500, credit $25,000, difference $3,500 -> REJECTED.

• `JE-005`: account `6315` absent from COA -> REJECTED, even though debit and credit both $18,500.

• `JE-008`: $320,000 debit and credit to the *same* intercompany account `2170` -> REVIEW, net zero; circularity cannot be proven without entity/counterparty metadata.

• Seven other journals pass the **implemented journal-level checks**; this does **not** mean they are approved to post. In particular, FX revaluation `JE-003` still requires external valuation support.

• Current TB has `9999` unmapped; account `6310` repeats twice in USD and `1110` repeats across USD/EUR/GBP (multi-currency repetition is allowed, same-currency duplicate requires investigation, not blind deletion).

• Missing GBP `period_end` FX rate; `1150` and `2170` cash-flow category `TBD`; `1290` empty header; `8000` is a parent typed `Expense` rather than `Header`; prior account `6905` absent from current COA.

• Raw current TB debit $71,072,200 vs credit $71,077,000 (difference -$4,800); **this is NOT a valid USD reconciliation because it mixes currencies**. Prior TB debit $35,177,800 vs credit $32,345,000 (difference $2,832,800) also mixes currencies and may have different scope. Do not invent a plug to balance either.

Failure matrix

Validation / self-correction loop

1. Validate file schemas and preserve original rows.

2. Emit structured findings with source pointers and severity.

3. Validate each journal independently; classify `ACCEPTED`, `REVIEW`, `REJECTED`.

4. Only a finance user can supply corrections or approve an exception. Each change is a **new version**, never an edit to raw source.

5. Re-run affected checks deterministically. Escalate after repeated unresolved exceptions; do not prompt-loop until numbers look plausible.

6. Release statements only if **all required** gates pass; an accepted journal is not a released statement.

Audit-grade traceability design

For each published cell, retain `statement_id, period, entity, currency, rule_version, formula, account_codes, normalized_ledger_row_ids, adjustment_line_ids, FX_rate_record_ids, COA_mapping_approval_ids, source_file_sha256, run_id`. Each source row retains filename, physical line number, raw values, original currency, applied rate and translation rule. A lineage graph supports cell -> accounts -> ledger rows -> original CSV/JSON and human approvals. An append-only event log records who approved what, when, and why. Prototype records file SHA-256 in diagnostics and per-entry SHA-256 traces, but **does not yet implement the**

full cell-lineage graph.

Production architecture and operations

Use PostgreSQL with immutable ingestion batches, versioned mapping tables, approval events and uniqueness constraints. Use object storage for raw evidence; background jobs with idempotent run keys and transactional checkpoints. Separate roles for uploader, preparer, reviewer and releaser; enforce tenant isolation, least privilege, PII-safe logs and retention policy. Instrument exception rates, mapping drift, reconciliation differences and reviewer turnaround. Test replay determinism, corrupted files, FX boundary dates, rounding, negative values, duplicate journal IDs and IC graph cycles. Never label generated financials 'audited' without an actual audit.

Scope / deliberate omissions

Only the adjustment validator is implemented end-to-end; `analyze_inputs.py` is supporting read-only diagnostics. No live LLM, posting service, statement builder, entity graph, or verified USD-translated TB in the prototype. This narrowness is intentional under the time budget.